

# Lab 10 — Divide, Conquer, and a Pivot

DSA27 Lab Manual · Week 10 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 10

## Contents

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Part 0 — Before you start                                          | 2  |
| Part 1 — <code>_merge</code> : two sorted runs become one          | 3  |
| Part 2 — <code>merge_sort_steps</code> and <code>merge_sort</code> | 4  |
| Part 3 — <code>_partition</code> : Lomuto's one pass               | 6  |
| Part 4 — <code>quick_sort_steps</code> : pivots and the call stack | 7  |
| Part 5 — <code>heap_sort_steps</code> : the array is the tree      | 9  |
| Part 6 — Count the comparisons                                     | 10 |
| Part 7 — Stability, and the test that cannot see it                | 12 |
| Part 8 — Measure the time: what are you timing?                    | 12 |
| Part 9 — Exercises at a glance                                     | 13 |
| Part 10 — Take-home practice (not graded)                          | 14 |
| Part 11 — Bridge to Lecture 11: trees                              | 14 |
| Summary                                                            | 15 |
| Answers to the checkpoints                                         | 15 |

**How to use this lab.** The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — this week that means a merge tree, a partition table and a heap drawn as a tree — and predict at each **Checkpoint** (answers at the end). Then write one function, run the tests named in its part, and only then move on. Three sorts, each built from one small helper: `_merge`, `_partition`, `_sift_down`. Get the helper right by hand and the sort is five lines. Parts 1–5 are the session; Parts 6–8 are for home.

---

|          |                                                    |
|----------|----------------------------------------------------|
| Duration | One 2-hour lab session, plus about 3 hours at home |
|----------|----------------------------------------------------|

**You will write**

the second half of `dsa/sorting.py` — `merge_sort_steps`, `quick_sort_steps`, `heap_sort_steps` and their plain forms, with the helpers `_merge`, `_choose_pivot`, `_partition` and `_sift_down`; and, for yourself, two small classes, `Counted` and `Card`

**Graded by**

`tests/test_sorting.py -k "merge or quick or heap"` (37 tests)

**Connects to**

Lecture 10 — Advanced sorting; Lab 09 (the `_steps` pattern); Lecture 03 (`merge_sorted`); and Week 12, where the heap returns as a priority queue

---

**What you will be able to do**

1. merge two sorted runs through a scratch Array by hand, and say which of two equal values goes first and why;
  2. write merge sort as a recursive generator, and explain what `yield from` does and what happens without it;
  3. trace Lomuto's partition as a table of `j`, boundary and the array, and state its invariant;
  4. write quicksort with four pivot strategies and a call stack that stays  $O(\log n)$  deep, and measure its  $O(n^2)$  worst case yourself;
  5. read an array as a tree, build a max-heap bottom-up, and write heap sort;
  6. count the comparisons each sort makes, and check stability with values that compare by key only;
  7. explain why timing your sorts can measure the snapshots instead of the sort — and fix it.
- 

## Part 0 — Before you start

### 0.1 Environment

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q # the environment c
pytest tests/test_sorting.py -q -k "merge or quick or heap" # 37 failures: noth
```

The 37 failures are all `NotImplementedError` — that is the starting point.

## 0.2 Prerequisites

- **Lab 09.** Your `bubble_sort_steps` and friends work, and you know the pattern: copy the input into an Array, yield a snapshot (`list(a)`, `highlight`) at each step, and write the plain form as “run the generator to the end, return the last state”. This week uses the same pattern, the same helper `_copy`, and the same `_finish` if you wrote one. If you did not, write the plain forms as in the docstring at the top of `dsa/sorting.py`.
- **Lecture 03.** Recursion, and `merge_sorted` in `dsa/recursion.py`. The merge step this week is the loop version of the same idea.
- **Generators (Lab 03).** `yield` makes a function a generator; calling it runs **nothing** until something iterates it. Part 2 depends on this.

## 0.3 Read the skeleton

Open `dsa/sorting.py` and read the second half: `merge_sort_steps`, `quick_sort_steps`, `heap_sort_steps`, and their plain forms. Each docstring gives the target cost. Two things to notice:

| Function                      | Signature                              | Note                                            |
|-------------------------------|----------------------------------------|-------------------------------------------------|
| <code>merge_sort_steps</code> | <code>(values)</code>                  | $O(n \log n)$ always, $O(n)$ extra              |
| <code>quick_sort_steps</code> | <code>(values, pivot="median3")</code> | pivot is "first", "last", "random" or "median3" |
| <code>heap_sort_steps</code>  | <code>(values)</code>                  | $O(n \log n)$ , $O(1)$ extra, not stable        |

The helpers — `_merge`, `_choose_pivot`, `_partition`, `_sift_down` — are not in the skeleton. You add them, above the functions that use them. A name that starts with `_` is private: the tests never call it, so you are free to shape it, but the lecture’s shapes are the ones this lab traces.

## 0.4 The storage rule, this week

Copy the input into the course Array, sort the Array, return a list — as in Lab 09. Merge sort’s scratch is an Array too. The median of three is computed with comparisons. Not allowed in `dsa/sorting.py`: `sorted()`, `list.sort()`, `heapq`, `bisect`, and slicing the Array (it raises `TypeError: Array does not support slicing anyway`). Python lists appear only as the snapshots you yield and the list you return.

---

## Part 1 — `_merge`: two sorted runs become one

Lecture 10, “Merging two sorted runs” and “`_merge`: the code”. You have written a merge twice already: in Week 2 (`merge_sorted` on two Arrays, `dsa/array_ops.py`) and in Week

3 (recursively, `dsa/recursion.py`). This one merges two **neighbouring** runs of the same Array, `a[lo:mid]` and `a[mid:hi]`, through a scratch Array, and copies the result back into `a`.

### 1.1 Draw it

Three indices: `i` walks the left run, `j` the right run, `k` the next free slot of scratch. Draw `a`, then scratch under it, and move the three arrows one step at a time.

**Checkpoint 1.** `a = [2, 5, 9, 1, 5, 6]`, `lo = 0`, `mid = 3`, `hi = 6`. Call the 5 at index 1 “5L” (left run) and the one at index 4 “5R”.

- (a) In what order are the six values written to scratch? Does 5L or 5R go first?
- (b) How many comparisons `a[i] <= a[j]` are made?
- (c) Which of the two “leftover” loops runs, and what does it copy?
- (d) If the test were `a[i] < a[j]`, what would change?

### 1.2 Write it

`_merge(a, lo, mid, hi, scratch)`, above `merge_sort_steps`:

1. Start `i` at `lo`, `j` at `mid`, `k` at `lo` — `scratch[k]` lines up with `a[k]`, so the merged range goes back to the same place.
2. While both runs have elements: compare the fronts; copy the smaller to `scratch[k]`; advance that run’s index and `k`. On a **tie, take from the left**: `<=`.
3. Copy whatever is left of the left run; then whatever is left of the right run. Only one of the two has anything.
4. Copy `scratch[lo:hi]` back into `a[lo:hi]` with a loop.

There is no test that calls `_merge` directly: its tests are merge sort’s, in Part 2. So check it by hand now, in the notebook:

```
from dsa.array import Array
from dsa.sorting import _merge
a = Array.from_values([2, 5, 9, 1, 5, 6])
_merge(a, 0, 3, 6, Array(6))
print(list(a))           # [1, 2, 5, 5, 6, 9]
```

---

## Part 2 — `merge_sort_steps` and `merge_sort`

Lecture 10, “`merge_sort_steps`: divide, conquer, combine”. One scratch Array, made once; an inner generator `sort(lo, hi)` that sorts `a[lo:hi]`; a snapshot after every merge.

## 2.1 Draw it

Draw the merge tree of the lecture's figure for your own input: split in the middle until each piece has one element, then merge neighbours on the way back up. Number the merges in the order the recursion does them — **left subtree completely first**.

**Checkpoint 2.** For `merge_sort_steps([6, 5, 3, 1, 8, 7, 2, 4])`:

- (a) List the merged ranges `a[lo:hi]` in the order the merges happen.
- (b) What is the array after the **third** merge? After the **sixth**?
- (c) How many frames does the generator yield in total, counting the first and the last?
- (d) How many comparisons does the whole sort make? (Count them merge by merge: a merge of two runs makes at most  $\text{len} - 1$ .)

## 2.2 Write it

1. `a = _copy(values)` and `scratch = Array(len(a))`; yield the first frame.
2. Inside, define `sort(lo, hi)`:
  - a range of 0 or 1 elements is sorted: return;
  - `mid = (lo + hi) // 2`;
  - sort the left half, sort the right half — each with **yield from**;
  - `_merge(a, lo, mid, hi, scratch)`, then yield `list(a)` with the merged range as the highlight, `tuple(range(lo, hi))`.
3. `yield from sort(0, len(a))`, then yield the last frame.
4. `merge_sort(values)`: run the generator to the end and return the last state.

Why `yield from` and not a plain call: `sort` contains `yield`, so `sort(lo, mid)` only **creates** a generator object and throws it away — nothing runs. `yield from sort(lo, mid)` runs it and passes its frames up.

## 2.3 Test it

```
pytest tests/test_sorting.py -v -k merge
```

Twelve tests: the eight fixed cases of `test_sorts_correctly`, “does not mutate the input”, 20 random lists, and the two generator tests — each with `merge_sort` in its name.

Then watch it, in `notebooks/10-sorting-advanced.ipynb`:

```
from viz.animate import step_slider
from dsa.sorting import merge_sort_steps
step_slider(merge_sort_steps([6, 5, 3, 1, 8, 7, 2, 4]))
```

## 2.4 When it fails

| Bug                                                                    | What you see                                                                                                                                                            | Fix                                                                                                               |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>sort(lo, mid)</code> without yield from                          | 5 tests fail, e.g. <code>assert [5, 2, 9, 1, 7] == [1, 2, 5, 7, 9]</code> — only the top-level merge ever runs                                                          | yield from <code>sort(lo, mid)</code> , and the same for the right half                                           |
| base case <code>if hi - lo &lt;= 0</code>                              | 11 tests fail with <code>RecursionError: maximum recursion depth exceeded — sort(lo, lo + 1) has mid = lo and calls itself on the same range for ever</code>            | a one-element range is already sorted: <code>hi - lo &lt;= 1</code>                                               |
| no copy back from scratch                                              | 7 tests fail, including <code>AssertionError: frames never change — are you yielding the same list?</code>                                                              | the last loop of <code>_merge</code> : <code>a[k] = scratch[k]</code> for <code>k</code> in <code>lo..hi-1</code> |
| the leftover loop for the left run is missing                          | 8 tests fail with <code>TypeError: '&lt;=' not supported between instances of 'int' and 'NoneType' — the unfilled scratch slots hold None and were copied into a</code> | both leftover loops                                                                                               |
| <code>&lt;</code> instead of <code>&lt;=</code> in <code>_merge</code> | <b>nothing fails</b>                                                                                                                                                    | see Part 7 — the tests cannot see it                                                                              |

## Part 3 — `_partition`: Lomuto's one pass

Lecture 10, “Lomuto's partition” and “`_partition`: the code”. The pivot is `a[hi]`. One index `j` scans `lo..hi-1`; one index boundary marks the end of the “smaller than the pivot” region.

**Invariant:** `a[lo:boundary] < pivot <= a[boundary:j]`.

### 3.1 Draw it

A table, one row per value of `j`, as in the lecture's figure:

|                |                   |                               |                    |                      |          |
|----------------|-------------------|-------------------------------|--------------------|----------------------|----------|
| <code>j</code> | <code>a[j]</code> | <code>a[j] &lt; pivot?</code> | <code>swap?</code> | array after the step | boundary |
|----------------|-------------------|-------------------------------|--------------------|----------------------|----------|

and a last row for the final swap of the pivot into `a[boundary]`.

**Checkpoint 3.** Trace `_partition(a, 0, 7)` on `a = [5, 8, 1, 9, 3, 7, 2, 6]` (the pivot is 6).

- (a) Fill in the table. Which values of `j` cause a swap?
- (b) What is the array at the end, and what does the function return?
- (c) How many comparisons with the pivot were made? Would that number change for a different order of the same eight values?

### 3.2 Write it

`_partition(a, lo, hi)` — both ends **inclusive**:

1. `pivot = a[hi]; boundary = lo.`
2. For `j` from `lo` to `hi - 1`: if `a[j] < pivot`, swap `a[boundary]` and `a[j]`, and add 1 to `boundary`.
3. Swap `a[boundary]` and `a[hi]`. Return `boundary`.

The strict `<` sends values **equal** to the pivot to the right. That is what the invariant says; keep it.

---

## Part 4 — `quick_sort_steps`: pivots and the call stack

Lecture 10, “Choosing the pivot” and “Stack depth: recurse on the smaller side”.

### 4.1 `_choose_pivot(a, lo, hi, strategy, rng)`

Return the **index** of the pivot in `a[lo..hi]`:

- `"first" → lo; "last" → hi;`
- `"random" → rng.randint(lo, hi)` — `rng` is a `random.Random` made once in `quick_sort_steps`, with a fixed seed so the frames are the same every run;
- `"median3" → whichever of lo, mid = (lo + hi) // 2 and hi holds the middle value of the three — with comparisons, no sorted();`
- anything else `→ ValueError`.

The caller swaps the chosen index with `hi`, so `_partition` always finds the pivot at the end and never needs to know how it was chosen.

### 4.2 Draw it: two ways to recurse

**Checkpoint 4.** Run in your head `quick_sort(list(range(8)), pivot="first")` — sorted input, the first element as pivot.

- (a) Where does each pivot end up, and how big are the two parts after each partition?
- (b) How many partitions, and how many comparisons in total? Give the formula for  $n$  elements.

- (c) Write the obvious version — partition, then **two** recursive calls. How deep does its call stack get on `list(range(n))`? At what  $n$  does Python stop it?
- (d) With “recurse into the smaller part, loop on the larger”, how deep does the stack get on the same input?

### 4.3 Write it

1. `a = _copy(values); rng = random.Random(27); yield the first frame.`
2. The inner `sort(lo, hi)` — inclusive range — is a **while** `lo < hi`: loop:
  - choose the pivot, swap it to `hi`, partition: `q` is its final index;
  - yield a snapshot with `(q,)` highlighted;
  - if the left part `lo..q-1` is the smaller one: yield from `sort(lo, q - 1)`, then `lo = q + 1` and go round again for the right part;
  - otherwise the mirror image: recurse on `q + 1..hi`, then `hi = q - 1`.
3. yield from `sort(0, len(a) - 1)`, and the last frame.
4. `quick_sort(values, pivot="median3")`: pass pivot through.

### 4.4 Test it

```
pytest tests/test_sorting.py -v -k quick
```

Thirteen tests: the twelve kinds above, plus `test_quick_sort_survives_its_worst_case_input`, which sorts `list(range(200))`.

### 4.5 When it fails

| Bug                                                                      | What you see                                                                                                                                             | Fix                                                                                               |
|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| no final swap in <code>_partition</code>                                 | 7 tests fail, e.g. <code>assert [1, 5, 2, 9, 7] == [1, 2, 5, 7, 9]</code>                                                                                | the pivot must go to <code>a[boundary]</code> before you return boundary                          |
| if <code>lo &lt; hi</code> : where the loop should be <code>while</code> | 4 tests fail, e.g. <code>assert [5, 2, 1, 7, 9] == ...</code> — the larger part is never sorted                                                          | <code>while lo &lt; hi</code> : — the loop <b>is</b> the second recursive call                    |
| the chosen pivot is never swapped to <code>hi</code>                     | <b>nothing fails</b> : every partition uses <code>a[hi]</code> , so all four strategies behave like “last”                                               | count comparisons (Part 6): “median3” on sorted input is suddenly as slow as “first” — $n(n-1)/2$ |
| two recursive calls instead of “smaller side, then loop”                 | <b>nothing fails</b> — 200 elements is under Python’s limit — but <code>quick_sort(list(range(1000)), "first")</code> raises <code>RecursionError</code> | recurse into the smaller part only                                                                |

The last two rows are why this part has a Checkpoint the tests cannot mark. Try the bad version once, on purpose, and see the `RecursionError` yourself.

## Part 5 — heap\_sort\_steps: the array is the tree

Lecture 10, “The array is the tree”, “Sift down” and “Build-heap in  $O(n)$ ”. The children of  $a[i]$  are  $a[2i + 1]$  and  $a[2i + 2]$ .

### 5.1 Draw it

Draw any list as a tree with the course’s tool — no code of yours needed:

```
from viz.draw import draw_array_as_tree
draw_array_as_tree([2, 9, 4, 7, 1, 8, 5, 3])
```

**Checkpoint 5.**  $v = [2, 9, 4, 7, 1, 8, 5, 3]$ .

- Draw it as a tree. Which indices are leaves? Which is the last parent?
- Build-heap sifts down indices 3, 2, 1, 0 in that order. Give the array after each sift.
- Then heap sort’s first step: swap  $a[0]$  with  $a[7]$  and sift down in  $a[0:7]$ . What is the array?
- Why does build-heap go from the last parent **down to 0**, and not from 0 up?

### 5.2 Write it

`_sift_down(a, index, size)` — the heap is  $a[0:size]$ :

- Loop: `largest = index`; compute left and right.
- If `left < size` and `a[left] > a[largest]`, `largest = left`. The same for right, comparing with `a[largest]` — the **larger** child.
- If `largest == index`, stop. Otherwise swap, and continue from `largest`.

`heap_sort_steps(values)`:

- `a = _copy(values)`, `n = len(a)`; yield the first frame.
- Build-heap: sift down every index from `n // 2 - 1` **down to 0**, with `size = n`. Yield a frame — the heap.
- For `end` from `n - 1` down to 1: swap `a[0]` and `a[end]`; sift down index 0 with `size = end`; yield with `(0, end)` highlighted.
- The last frame.

### 5.3 Test it

```
pytest tests/test_sorting.py -v -k heap
```

Twelve tests. Then all 37:

```
pytest tests/test_sorting.py -v -k "merge or quick or heap"
```

## 5.4 When it fails

| Bug                                                       | What you see                                                                                               | Fix                                                                                                        |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| build-heap goes <b>up</b> , for<br>index in range(n // 2) | only <b>2</b> tests fail: sorted<br>input [1, 2, 3, 4, 5]<br>gives [1, 2, 4, 5, 3],<br>and one random list | sifting a parent assumes its<br>subtrees are already heaps:<br>go from the last parent<br><b>down</b> to 0 |
| build-heap stops at 1,<br>range(n // 2 - 1, 0,<br>-1)     | 5 tests fail, e.g. [2, 3, 4,<br>5, 1] — the root was never<br>sifted                                       | the stop value of range is<br>excluded: -1                                                                 |
| sift down with size = n in<br>the sort loop               | 7 tests fail; the output is<br>often <b>reversed</b> , e.g. [5, 4,<br>3, 2, 1]                             | the sorted tail is not part of<br>the heap: size = end                                                     |
| only the left child compared                              | 6 tests fail, e.g. [1, 3, 2,<br>4, 5]                                                                      | compare with <b>both</b> children;<br>move the larger up                                                   |
| the sort loop stops at 2,<br>range(n - 1, 1, -1)          | 7 tests fail, e.g. [2, 1]<br>stays [2, 1] — the last two<br>are never ordered                              | range(n - 1, 0, -1)                                                                                        |

The first row is the sneaky one: 35 of 37 tests pass. If your heap sort passes “almost everything”, check the direction of the build loop first.

## Part 6 — Count the comparisons

Lecture 10, “Measured”. Lab 08 counted how many elements a search **reads** with a class that pretends to be a list. This week you count **comparisons** with a class that pretends to be a number. Not graded; write it in the notebook, not in dsa/.

### 6.1 Write Counted

```
class Counted:
    comparisons = 0  # one counter, shared by every Counted

    def __init__(self, value):
        self.value = value

    def __lt__(self, other):
        Counted.comparisons += 1
```

```

    return self.value < other.value
    # ... and the same for __le__, __gt__ and __ge__

```

Your sorts use `<`, `<=` and `>`; Python turns each into a call of the matching method, so the sorts cannot tell a `Counted` from a number. Then:

```

def comparisons(sort, values, *args):
    items = [Counted(v) for v in values]
    Counted.comparisons = 0
    result = sort(items, *args)
    assert [c.value for c in result] == sorted(values)
    return Counted.comparisons

```

(`tools/figures_l10.py` has the course's version of both. Look at it after writing yours.)

## 6.2 The pivot experiment

**Checkpoint 6.** Predict, **then** run:

```

for n in [100, 200, 400]:
    print(n, [comparisons(quick_sort, list(range(n)), p)
              for p in ["first", "median3", "random"]])

```

- (a) Give the exact numbers for "first". What happens to them each time  $n$  doubles?
- (b) Roughly what size do you expect for "median3" — closer to  $n$ ,  $n \log_2 n$  or  $n^2$ ?
- (c) Replace `quick_sort` by `merge_sort` and by `heap_sort` on the same sorted lists. Which one does **fewer** comparisons than on random data, and why?

With the reference solution the "median3" counts are 606, 1,407 and 3,208, and "random" 642, 1,533 and 3,743 — a little over  $n \log_2 n$  (664, 1,529, 3,458). Yours may differ by a few percent if your median-of-three compares in a different order; the shape may not.

## 6.3 Build-heap is linear — count it

```

import random
from dsa.array import Array
from dsa.sorting import _sift_down

for n in [1000, 2000, 4000, 8000]:
    items = Array.from_values([Counted(x) for x in random.Random(3).sample(range(
    Counted.comparisons = 0
    for index in range(n // 2 - 1, -1, -1):
        _sift_down(items, index, n)
    print(n, Counted.comparisons, round(Counted.comparisons / n, 2))

```

The reference gives 1,860, 3,712, 7,510 and 15,028 comparisons: about **1.87  $n$** , the same ratio at every size. An  $O(n \log n)$  build would show the ratio growing. That is Lecture 10's sum  $n(1/4 + 2/8 + 3/16 + \dots)$ , measured.

## Part 7 — Stability, and the test that cannot see it

Lecture 10, “Stability”. A sort is stable if equal keys keep their input order. `tests/test_sorting.py` checks stability for insertion sort only (`test_insertion_sort_is_stable`), and none of the 37 tests of this lab checks it at all: they sort plain numbers, and two equal numbers cannot be told apart. Nor would **tuples** help — Python compares `(1, "a")` and `(1, "c")` by their second items, so any correct sort puts them in that order. To see stability, you need values that compare by key **only**, the way the insertion-sort test’s `Card` class does.

### 7.1 Write Card

```
class Card:
    def __init__(self, key, tag):
        self.key, self.tag = key, tag
    def __lt__(self, other):
        return self.key < other.key
    # ... __le__, __gt__, __ge__ likewise — the tag is never compared
    def __repr__(self):
        return f"{self.key}{self.tag}"
```

**Checkpoint 7.** `cards = [Card(2, "a"), Card(1, "b"), Card(2, "c"), Card(1, "d")]`.

- (a) Predict `merge_sort(cards)`, `quick_sort(cards)` and `heap_sort(cards)`. Then run them.
- (b) Change the `<=` in your `_merge` to `<`. Which of the 37 tests fail? What does `merge_sort(cards)` give now? Put the `<=` back.
- (c) Write a test of your own — in a file outside `tests/`, or in the notebook — that fails for the `<` version and passes for the `<=` one.

This is the lesson of Lab 08’s `lo < hi` bug again, from the other side: a test suite is a set of examples, and a property no example exercises is a property nobody has checked.

---

## Part 8 — Measure the time: what are you timing?

Lecture 10, “Measured”, last paragraph. Time your `merge_sort` in the notebook:

```
import random, time
from dsa.sorting import merge_sort

for n in [1000, 2000, 4000]:
    values = [random.random() for _ in range(n)]
```

```
start = time.perf_counter()
merge_sort(values)
print(n, round(time.perf_counter() - start, 3))
```

### Checkpoint 8.

- If merge sort is  $O(n \log n)$ , by roughly what factor should the time grow from  $n = 1,000$  to  $2,000$ ? What factor do you **see**?
- Find the line that costs  $O(n)$  and runs  $n - 1$  times. (Hint: it is not in `_merge`.)
- Fix it without breaking the animation or the tests.

With the reference code as the lecture shows it, this printed about 0.36, 1.29 and 4.7 seconds — four times as long for twice the input:  $n^2$ , not  $n \log n$ . The sort is fine; the **snapshots** are not free. Fix it the way the basic sorts of the reference solution do: give the `_steps` form a parameter, `snapshot=list`, write `yield snapshot(a), ...` instead of `yield list(a), ...`, and let the plain form pass a function that returns a itself, uncopied. The animation still gets fresh lists; `merge_sort` gets none, and only its last state is turned into a list. After the fix, the same loop printed about 0.07, 0.13 and 0.27 seconds — twice the time for twice the input, plus a little: the  $\log n$ . Apply the same fix to `quick_sort` and `heap_sort`, and run the 37 tests again.

## Part 9 — Exercises at a glance

| Function                                              | Target cost                                              | The trap                                                                                          | -k filter            |
|-------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------|
| <code>_merge</code>                                   | $O(hi - lo)$                                             | $\leq$ for stability; both leftover loops; copy back                                              | merge (12)           |
| <code>merge_sort_steps / merge_sort</code>            | $O(n \log n)$ , $O(n)$ extra                             | <code>yield from</code> ; base case $hi - lo \leq 1$ ; one scratch Array                          | merge (12)           |
| <code>_partition</code><br><code>_choose_pivot</code> | $O(hi - lo)$<br>$O(1)$                                   | the final swap; strict $<$ return an <b>index</b> ; median of three without <code>sorted()</code> | quick (13)<br>Part 6 |
| <code>quick_sort_steps / quick_sort</code>            | $O(n \log n)$ average, $O(n^2)$ worst, $O(\log n)$ stack | swap the pivot to $hi$ ; while, smaller side first                                                | quick (13)           |
| <code>_sift_down</code>                               | $O(\log n)$                                              | the <b>larger</b> child; size bounds the heap                                                     | heap (12)            |
| <code>heap_sort_steps / heap_sort</code>              | $O(n \log n)$ , $O(1)$ extra                             | build <b>down</b> from $n // 2 - 1$ ; sort with <code>size = end</code>                           | heap (12)            |

All 37 at once, and then the whole file with Lab 09's sorts:

```
pytest tests/test_sorting.py -v -k "merge or quick or heap"
pytest tests/test_sorting.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing, and nothing in `dsa/sorting.py` may call `sorted`, `.sort`, `heapq` or `bisect`.

---

## Part 10 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week10-questions.md`, with answers in `week10-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week10.py`: `merge_k_sorted`, `count_inversions`, `kth_smallest`, `sort_colours` and `merge_sort_bottom_up` (W10-C1 to W10-C5). Each one is a piece of this week's sorts put to another use:

```
pytest tests/test_practice_week10.py -v
```

Hint for `count_inversions`: it is merge sort, plus one addition inside the merge.

2. **W10-T2** — `trace_partition` on `[5, 8, 1, 9, 3, 7, 2, 6]`: Homework 10, item 2, and Checkpoint 3 of this lab.
3. **W10-S1** — draw the merge tree of `[6, 5, 3, 1, 8, 7, 2, 4]`, every level.
4. **W10-B2** — a heap sort that builds its heap from the top. Find an input on which it fails, without running it.
5. **W10-K2** — the recurrence of quicksort with the first element as pivot, on sorted input. Solve it.

The worked solutions are in `solutions/dsa/sorting.py` and `solutions/practice/week10.py` — for after you have tried. `pytest --solutions tests/test_practice_week10.py` runs the tests on them.

---

## Part 11 — Bridge to Lecture 11: trees

Two things from this lab come back next week.

**The pivot story.** Insert the values `1, 2, 3, . . . , n` into a binary search tree, one at a time: each new value is larger than all the others, so it goes to the right of the right of the right... The tree becomes a chain `n` deep — exactly the shape of quicksort with `pivot="first"` on sorted input, which you drew in Checkpoint 4. Same input, same disaster, different structure.

**Sorting for free.** A binary search tree keeps every left subtree smaller than its root and every right subtree at least as large. Visit the left subtree, then the root, then the right subtree — an **in-order traversal** — and the values come out sorted. Building the tree

is quicksort's partition done one value at a time; reading it in order is the recursion that follows. Lecture 11 makes this precise.

And the heap you drew in Part 5 is a tree stored in an array, with no pointers. Week 11's trees have nodes and pointers; Week 12 returns to the array.

---

## Summary

| Idea                         | The one line to keep                                                                                                  |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Divide and conquer<br>_merge | Split, solve the parts recursively, combine.<br>Compare fronts, copy the smaller; $\leq$ takes from the left: stable. |
| Merge sort                   | yield from both halves, then merge; $O(n \log n)$ always, $O(n)$ extra.                                               |
| Lomuto partition             | $a[lo:boundary] < pivot \leq a[boundary:j]$ ; one final swap.                                                         |
| Pivot                        | "first" on sorted input: $n(n-1)/2$ comparisons, measured.                                                            |
| Stack depth                  | Recurse into the smaller part, loop on the larger: $O(\log n)$ .                                                      |
| Heap                         | Children of $i$ at $2i+1$ and $2i+2$ ; every parent $\geq$ its children.                                              |
| Build-heap                   | From the last parent <b>down</b> to 0: about $1.9n$ comparisons — $O(n)$ .                                            |
| Heap sort                    | Swap the max to the back, sift down with $size = end$ .                                                               |
| Stability                    | Tuples cannot test it; compare by key only (Card).                                                                    |
| Timing                       | A snapshot costs $O(n)$ ; the plain form must not copy.                                                               |

---

## Answers to the checkpoints

**Checkpoint 1.** (a) 1, 2, **5L**, 5R, 6, 9. 5L goes first: when  $a[i] = 5L$  and  $a[j] = 5R$ ,  $5 \leq 5$  is true, so the left one is taken. (b) **5**: 2 vs 1, 2 vs 5, 5 vs 5, 9 vs 5, 9 vs 6. Then the right run is empty. (c) The left leftover loop copies 9. The right one does nothing. (d)  $5 < 5$  is false, so 5R would be taken before 5L. The numbers come out the same — [1, 2, 5, 5, 6, 9] — but equal elements have swapped: the merge is no longer stable.

**Checkpoint 2.** (a)  $a[0:2]$ ,  $a[2:4]$ ,  $a[0:4]$ ,  $a[4:6]$ ,  $a[6:8]$ ,  $a[4:8]$ ,  $a[0:8]$ . (b) After the third merge: [1, 3, 5, 6, 8, 7, 2, 4]. After the sixth: [1, 3, 5, 6, 2, 4,

7, 8]. (c) **9**: the first frame, one per merge ( $n - 1 = 7$ ), and the last. (d) **14**, counted with the reference. Merge by merge: 1, 1, 2, 1, 1, 2, 6. For example, merging [5, 6] with [1, 3] takes 2 (5 vs 1, 5 vs 3), and then the right run is empty, so 5 and 6 are copied without a comparison. The last merge, [1, 3, 5, 6] with [2, 4, 7, 8], takes 6. Always at most  $n \log_2 n = 24$ .

### Checkpoint 3.

| j   | a[j] | < 6?                 | array after the step     | boundary         |
|-----|------|----------------------|--------------------------|------------------|
| 0   | 5    | yes: swap a[0], a[0] | [5, 8, 1, 9, 3, 7, 2, 6] | 1                |
| 1   | 8    | no                   | [5, 8, 1, 9, 3, 7, 2, 6] | 1                |
| 2   | 1    | yes: swap a[1], a[2] | [5, 1, 8, 9, 3, 7, 2, 6] | 2                |
| 3   | 9    | no                   | [5, 1, 8, 9, 3, 7, 2, 6] | 2                |
| 4   | 3    | yes: swap a[2], a[4] | [5, 1, 3, 9, 8, 7, 2, 6] | 3                |
| 5   | 7    | no                   | [5, 1, 3, 9, 8, 7, 2, 6] | 3                |
| 6   | 2    | yes: swap a[3], a[6] | [5, 1, 3, 2, 8, 7, 9, 6] | 4                |
| end |      | swap a[4], a[7]      | [5, 1, 3, 2, 6, 7, 9, 8] | returns <b>4</b> |

- (a)  $j = 0, 2, 4$  and 6. (The swap at  $j = 0$  swaps  $a[0]$  with itself — harmless.)  
 (b) [5, 1, 3, 2, 6, 7, 9, 8], and it returns 4: the pivot's final index.  
 (c) **7** — one per non-pivot element,  $hi - lo$ . It never depends on the order: the loop always runs from  $lo$  to  $hi - 1$ . Only the number of **swaps** does.

**Checkpoint 4.** (a) Each pivot is the smallest value of its range, so it stays where it is: at index 0, then 1, then 2, ... The left part is always **empty**, the right part has everything else.  
 (b) **7** partitions (the reference yields frames highlighting 0, 1, ..., 6), and  $7 + 6 + \dots + 1 = \mathbf{28}$  comparisons. For  $n$  elements:  $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$ . (c) One frame per partition:  $n$  deep. Python's limit is 1,000 frames, and each level of a `yield` from chain uses a frame: with the reference's partition and two recursive calls, `list(range(500))` still sorts and `list(range(1000))` raises `RecursionError`. (d) The smaller part is always the empty one, so every recursive call returns at once: the stack is **one** extra frame deep, whatever  $n$  is. The loop does the rest. The time is still  $n(n - 1)/2$  comparisons.

**Checkpoint 5.** (a) Indices 4 to 7 (values 1, 8, 5, 3) are leaves; the last parent is index 3 ( $n // 2 - 1$ ), whose only child is index 7. (b)

| sift                       | array after                                                                      |
|----------------------------|----------------------------------------------------------------------------------|
| index 3 (7; child 3)       | [2, 9, 4, 7, 1, 8, 5, 3] — $7 \geq 3$ , no change                                |
| index 2 (4; children 8, 5) | [2, 9, 8, 7, 1, 4, 5, 3]                                                         |
| index 1 (9; children 7, 1) | [2, 9, 8, 7, 1, 4, 5, 3] — no change                                             |
| index 0 (2; children 9, 8) | [9, 7, 8, 3, 1, 4, 5, 2] — 2 sinks<br>three levels: swaps with 9, then 7, then 3 |

- (c) Swap 9 and 2: [2, 7, 8, 3, 1, 4, 5, 9]; sift 2 down in  $a[0:7]$ : it swaps with 8 (the larger child), then with 5: [8, 7, 5, 3, 1, 4, 2, 9].

- (d) `_sift_down` assumes both subtrees of the node are already heaps. Going from the last parent **down** to the root, every subtree below has been sifted before its parent. Going **up** from 0, the root is sifted while its subtrees are still unordered, and a large value deep in the tree never reaches the top.

**Checkpoint 6.** (a) 4,950, 19,900 and 79,800 — exactly  $n(n-1)/2$ . Doubling  $n$  multiplies them by about **4**: quadratic. (b) Close to  $n \log_2 n$  (664, 1,529 and 3,458): the reference gives 606, 1,407 and 3,208 for "median3". (c) **Merge sort**: 316, 732 and 1,664 on sorted input, about half of  $n \log_2 n$  — in every merge the whole left run is smaller, so it empties after half as many comparisons and the right run is copied without any. Heap sort does **not** gain (1,081, 2,587 and 5,984): building a heap from sorted input moves every small value down, and the sort phase is the same.

**Checkpoint 7.** (a) `merge_sort`: [1b, 1d, 2a, 2c] — stable. `quick_sort`: [1b, 1d, 2c, 2a]. `heap_sort`: [1b, 1d, 2c, 2a]. Both swap the two 2s. (b) **None of the 37 fails**: they all sort plain numbers, where two equal values are indistinguishable. `merge_sort(cards)` now gives [1d, 1b, 2c, 2a]: both pairs reversed. (c) For example:

```
cards = [Card(2, "a"), Card(1, "b"), Card(2, "c"), Card(1, "d")]
assert [c.tag for c in merge_sort(cards)] == ["b", "d", "a", "c"]
```

**Checkpoint 8.** (a) Expected: a little over **2** ( $2,000 \log 2,000 / 1,000 \log 1,000 \approx 2.2$ ). Seen: about **4** — 0.36, 1.29, 4.7 seconds. (b) `yield list(a), tuple(range(lo, hi))` after each merge: `list(a)` copies all  $n$  values, and there are  $n-1$  merges —  $O(n^2)$  copying, which dwarfs the  $O(n \log n)$  sort. (c) A snapshot parameter, `list` by default for the animation, and a no-copy function for the plain form (Part 8). The tests still pass — they call the `_steps` forms with the default — and the time now roughly doubles with  $n$ .